

ObRail Europe

Rapport Technique — Développement de Modèles d'Intelligence Artificielle

MSPR Bloc E6.2 / E6.4 — RNCP36581 Développeur IA & Data Science

EPSI — Année 2025-2026

Équipe projet : Messa · Adam · Akram · Aymen

Dépôt GitHub : <https://github.com/Meech667/obrail-europe>

Dataset : <https://drive.google.com/file/d/1vt7VH9y0hb1HF0VJOSGWkntRVcbhafPr/view?usp=sharing>

1. Contexte du projet

ObRail Europe est un observatoire indépendant spécialisé dans le ferroviaire et la mobilité durable, créé en 2018. L'organisation s'est imposée comme un acteur de référence dans l'analyse des flux ferroviaires européens et la promotion du transport bas-carbone.

1.1 Mission d'ObRail Europe

- Collecter et analyser les données relatives aux dessertes ferroviaires en Europe
- Évaluer l'impact environnemental des modes de transport longue distance
- Produire des études comparatives destinées aux décideurs politiques et aux opérateurs ferroviaires
- Accompagner la transition écologique en favorisant l'usage du train comme alternative à l'avion sur les trajets intra-européens

1.2 Partenaires et cadre stratégique

ObRail travaille en partenariat avec les institutions européennes (Commission européenne, Parlement européen), les ONG environnementales Transport & Environnement et Back-on-Track, ainsi qu'avec les opérateurs SNCF, ÖBB Nightjet, DB et Trenitalia.

Le projet s'inscrit dans deux grandes stratégies européennes : le Green Deal européen (neutralité carbone à l'horizon 2050) et le programme TEN-T (Trans-European Transport Network), qui vise à renforcer l'intégration et la compétitivité du réseau ferroviaire en Europe.

1.3 Objectifs de la MSPR

ObRail souhaite développer un ensemble de modèles d'IA capables de répondre à deux enjeux métiers complémentaires à partir de données ferroviaires européennes. L'objectif est de constituer un référentiel de données fiable et harmonisé qui servira à l'analyse de la couverture ferroviaire, à l'entraînement de modèles prédictifs, et au développement d'un service applicatif destiné aux partenaires institutionnels.

Enjeu	Type de modèle	Variable cible	Responsable
Estimer les émissions CO2 futures selon des scénarios de développement réseau	Régression	kgCO2_futur (valeur continue)	Équipe
Identifier les liaisons candidates à remplacer un vol aérien	Classification	substitution_avion (0 ou 1)	Équipe

2. Présentation des données

2.1 Source et constitution du dataset

Le dataset eu_trips_v2.csv a été constitué lors de la phase ETL (Bloc E6.1) à partir de sources open data européennes : portails GTFS nationaux (transport.data.gouv.fr, DB Open Data, Renfe), Eurostat pour les facteurs d'émission, et ADEME pour les facteurs carbone. Il contient 142 420 trajets ferroviaires couvrant quatre pays : France, Espagne, Italie et Allemagne.

Le dataset est disponible à l'adresse suivante :

<https://drive.google.com/file/d/1vt7VH9y0hb1HF0VJOSGWkntRVcbhafPr/view?usp=sharing>

2.2 Description des colonnes

Colonne	Description	Type
distance_km	Distance géographique calculée entre les gares (formule haversine)	Numérique
duration_minutes	Durée du trajet en minutes	Numérique
n_stops	Nombre d'arrêts intermédiaires	Numérique
departure_minutes / arrival_minutes	Heures de départ et d'arrivée en minutes depuis minuit	Numérique
country	Code pays : FR, ES, IT, DE	Catégorielle
type_train	Type de traction : electric / diesel	Catégorielle
consommation_energy	Consommation énergétique en kWh/km	Numérique
gco2_per_kwh	Facteur carbone du réseau électrique du pays (gCO2/kWh)	Numérique
consommation_totale	Consommation totale calculée en kWh	Numérique
co2_estime	Émissions CO2 estimées en gCO2	Numérique
origin_stop_lat/lon	Coordonnées GPS de la gare d'origine	Numérique
destination_stop_lat/lon	Coordonnées GPS de la gare de destination	Numérique

2.3 Statistiques descriptives clés

Variable	Minimum	Médiane	Maximum	Observations
distance_km	20 km	48 km	938 km	Majorité de courts trajets régionaux
duration_minutes	0 min	70 min	1425 min	Quelques trajets très longs (outliers réels)
n_stops	2	12	202	Réseau dense avec nombreux arrêts
co2_estime	4 362 gCO2	120 670 gCO2	6 519 651 gCO2	Forte variabilité selon la distance
consommation_totale	200 kWh	687 kWh	18 758 kWh	Corrélée à 0.99 avec distance_km

2.4 Qualité des données

- 142 420 lignes sans valeurs manquantes sur les variables retenues pour la modélisation
- Seule la colonne `route_long_name` présente 44.46% de valeurs manquantes — colonne exclue de toute analyse
- Forte colinéarité entre `distance_km` et `consommation_totale` (corrélation = 0.99) — la consommation est calculée directement depuis la distance
- Déséquilibre géographique : l'Allemagne représente le plus grand volume de trajets, l'Italie est sous-représentée dans les liaisons longue distance

2.5 Feature engineering

Plusieurs variables dérivées ont été créées pour enrichir la description de chaque trajet :

- `vitesse_moyenne_kmh` : rapport distance / durée — indicateur de la qualité de service de la ligne
- `co2_par_km` : intensité carbone du trajet — variable normalisée par la distance
- `is_diesel` / `is_electric` : indicateurs binaires de traction — complémentaires à `type_train`
- `stops_per_100km` : nombre d'arrêts rapporté à la distance — mesure de la densité du service

3. Cadrage et exploration des données

Cette phase de cadrage constitue le socle analytique du projet. Elle présente les problématiques retenues, les scénarios métier construits et les visualisations exploratoires qui ont guidé les choix de modélisation.

3.1 Problématiques retenues

Problématique 1 — Régression : Comment estimer les émissions futures d'un trajet ferroviaire (kgCO₂_futur) à partir de ses caractéristiques opérationnelles et d'un scénario d'amélioration du réseau ?

Problématique 2 — Classification : Comment repérer automatiquement les liaisons ferroviaires candidates à la substitution avion → train parmi l'ensemble des trajets disponibles ?

Ces deux problématiques sont cohérentes avec les données disponibles, elles permettent de couvrir deux familles de modèles différentes (régression et classification), et elles répondent directement aux enjeux stratégiques d'ObRail Europe.

3.2 Scénarios métier

Quatre scénarios ont été construits et appliqués à chaque trajet pour simuler différentes politiques de développement du réseau ferroviaire européen. Ces scénarios permettent de comparer l'effet des politiques sur un même portefeuille de liaisons.

Scénario	Description	Règle appliquée
reference	Aucune évolution — baseline actuel	Aucune modification des données
diesel_50_electrique	Électrification partielle des trains diesel à 50%	consommation × 0.50 pour les trains diesel
conso_moins_15	Optimisation énergétique générale sur tous les trains	consommation × 0.85 pour tous les trains
distance_moins_10	Nouvelles lignes directes réduisant les trajets de 10%	distance × 0.90 et consommation × 0.90

3.3 Principaux enseignements de l'EDA

- La majorité des trajets sont des liaisons régionales courtes (médiane 48 km) — seuls 8.4% entrent dans le corridor de substitution avion/train
- La France dispose d'un réseau TGV adapté aux distances intermédiaires — elle concentre la majorité des liaisons substituables
- Les trains électriques dominent dans la classe substituable — cohérent avec la réalité des grandes lignes européennes (TGV, AVE, Frecciarossa, ICE)
- La forte corrélation distance_km / consommation_totale (0.99) crée une circularité avec les deux variables cibles construites — limite identifiée et documentée

4. Enjeu 1 — Régression CO2

4.1 Problématique

Estimer les émissions de CO2 futures d'une liaison ferroviaire selon différents scénarios de développement du réseau. C'est un problème de régression : prédire une valeur continue (kgCO2_futur) à partir des caractéristiques d'une liaison et d'un scénario.

4.2 Construction de la variable cible

La colonne kgCO2_futur n'existait pas dans le dataset original. Elle a été construite par simulation à partir des émissions actuelles, modifiées selon le scénario appliqué. Cette approche de labellisation par règles de simulation est cohérente avec l'absence de données terrain mesurées.

Les émissions actuelles sont calculées différemment selon le type de traction : pour les trains électriques, $\text{consommation_totale} \times \text{gco2_per_kwh} / 1000$; pour les trains diesel, une formule empirique basée sur la consommation de carburant.

4.3 Préparation des données

Features retenues

- Variables numériques : distance_scenario_km, duration_minutes, n_stops, consommation_energy, gco2_per_kwh, consommation_scenario, vitesse_moyenne_kmh, co2_par_km, is_diesel, is_electric
- Variables catégorielles : type_train, scenario

Pipeline de prétraitement

Un Pipeline sklearn a été construit avec un ColumnTransformer embarquant deux transformations :

- StandardScaler sur les 10 variables numériques — centrage ($\mu=0$) et réduction ($\sigma=1$) pour éviter que les grandes valeurs de consommation n'écrasent les petites
- OneHotEncoder sur type_train et scenario — conversion en colonnes binaires

L'intégration dans un Pipeline garantit que le fit du préprocesseur se fait uniquement sur le jeu d'entraînement, sans fuite d'information vers la validation ou le test.

Split 70 / 15 / 15

- Train (70%) : entraînement des modèles — 99 694 lignes
- Validation (15%) : comparaison et sélection du meilleur modèle — 21 363 lignes
- Test (15%) : évaluation finale non biaisée, jamais vu pendant les étapes précédentes — 21 363 lignes

4.4 Trois modèles candidats

Modèle 1 — Régression Linéaire (baseline)

Modèle le plus simple : suppose une relation linéaire entre les features et le CO2 futur. Sert de référence pour mesurer le gain des modèles plus complexes. Très interprétable via ses coefficients, mais limité si les relations entre variables sont non-linéaires ou avec interactions.

Modèle 2 — Random Forest Regressor

Ensemble de 100 arbres de décision entraînés en parallèle sur des sous-échantillons aléatoires (bagging). Capture les relations non-linéaires, robuste aux valeurs aberrantes, moins sujet au surapprentissage qu'un arbre seul. Bonne référence pour les données tabulaires — pas besoin de normalisation pour les arbres.

Modèle 3 — XGBoost Regressor

Boosting gradient : les arbres sont construits séquentiellement, chaque arbre corrige les erreurs du précédent. Algorithme de référence sur les données tabulaires — gère bien les relations complexes entre variables énergétiques et CO2. Plus sensible aux hyperparamètres que Random Forest, d'où l'optimisation par GridSearchCV.

4.5 Comparaison des modèles sur le jeu de validation

Modèle	MAE (kgCO2)	RMSE (kgCO2)	R ²	Famille
Régression Linéaire	~1.5	~3.8	~0.998	Linéaire (baseline)
Random Forest	~0.8	~2.1	~0.999	Bagging
XGBoost	~0.9	~2.6	1.000	Boosting — retenu

XGBoost obtient le meilleur R² et les MAE / RMSE les plus faibles sur la validation — il capture mieux les relations non-linéaires entre les variables énergétiques et le CO2 futur. Random Forest est une alternative solide. La régression linéaire reste utile comme baseline interprétable mais sous-performe sur ce jeu multi-scénarios.

4.6 Optimisation par GridSearchCV

Le modèle XGBoost a été optimisé via GridSearchCV avec validation croisée à 5 folds sur le jeu d'entraînement. 18 combinaisons × 5 folds = 90 entraînements au total.

Hyperparamètre	Valeurs testées	Meilleure valeur
n_estimators	100, 200	200
max_depth	3, 5, 7	7
learning_rate	0.05, 0.1, 0.2	0.2

4.7 Cross-validation — Robustesse du modèle

La cross-validation à 5 folds évalue la stabilité du modèle sur l'ensemble des données d'entraînement. Le dataset est découpé en 5 parties égales — le modèle est entraîné 5 fois, chaque fois sur 4 parties et testé sur la 5ème restante. Le score R² final est la moyenne des 5 scores obtenus. Un écart-type faible confirme que le modèle généralise bien au-delà du jeu d'entraînement utilisé.

Métrique	Valeur
Meilleur R ² (GridSearchCV CV)	1.0000
R ² moyen cross-validation 5 folds	1.0000
Écart-type cross-validation	0.0001
Conclusion	✓ Modèle stable — faible variance entre les folds

4.8 Résultats finaux sur le jeu de test

Évaluation du modèle optimisé sur X_{test} / y_{test} — données mises de côté depuis le début, jamais utilisées ni pour l'entraînement ni pour la sélection. C'est la seule mesure non biaisée des performances réelles.

Métrique	Résultat	Interprétation
MAE	0.939 kgCO ₂	Erreur moyenne inférieure à 1 kgCO ₂ — très faible
RMSE	2.625 kgCO ₂	Les grosses erreurs restent limitées
R^2	1.0000	Le modèle explique la quasi-totalité de la variance

Un MAE de 0.939 kgCO₂ signifie qu'en moyenne, le modèle se trompe de moins de 1 kg sur ses estimations. Pour ObRail, cela représente une marge d'erreur suffisamment faible pour orienter des décisions stratégiques — priorisation des scénarios, argumentation auprès des ministères des Transports européens.

4.9 Feature importance

Les variables les plus importantes pour XGBoost sont les variables énergétiques liées au scénario (consommation_scenario, gco2_per_kwh) — cohérent avec la construction de la variable cible. La distance_scenario_km et la vitesse_moyenne_kmh contribuent également significativement.

5. Enjeu 2 — Classification substitution avion/train

5.1 Problématique

Identifier automatiquement quelles liaisons ferroviaires sont candidates à remplacer un vol aérien intra-européen. C'est un problème de classification binaire : substituable (1) ou non (0).

La variable cible n'existait pas dans les données — elle est construite par règles métier issues des études Transport & Environnement et Back-on-Track (2021).

5.2 Construction de la variable cible

Deux conditions doivent être réunies pour qu'une liaison soit considérée substituable :

- distance_km entre 300 et 1500 km — corridor où le train peut concurrencer l'avion (en dessous de 300 km personne ne prend l'avion, au-dessus de 1500 km le temps de trajet devient rédhibitoire)
- duration_minutes < 480 min (8 heures) — seuil d'acceptabilité pour un voyageur d'affaires ou de loisir

substitution_avion = 1 si les deux conditions sont remplies, 0 sinon. C'est une labellisation par règles, pas une observation terrain — voir section Limites.

Classe	Effectif	Pourcentage
Non substituable (0)	130 470	91.61%
Substituable (1)	11 950	8.39%

Ce déséquilibre de classes (91.6% / 8.4%) est géré via `class_weight='balanced'` pour la Logistic Regression et `scale_pos_weight` (ratio ≈ 10.92) pour XGBoost. Le F1-score macro est préféré à l'accuracy comme métrique principale.

5.3 Préparation des données

5.3.1 Sélection des features

Features retenues : distance_km, duration_minutes, n_stops, co2_estime, consommation_totale, type_train, country.

Colonnes exclues :

- Coordonnées GPS : redondantes avec distance_km (calculée par haversine depuis ces mêmes coordonnées)
- arrival_minutes, departure_minutes : non pertinents pour une décision de substitution
- route_id, trip_id, service_id : identifiants sans pouvoir prédictif
- gco2_per_kwh, consommation_energy : déjà incorporés dans co2_estime et consommation_totale

5.3.2 Encodage des variables catégorielles

LabelEncoder retenu plutôt que OneHotEncoding : type_train a 2 modalités, country en 4 — le surcoût du One-Hot est négligeable mais inutile. Pour les arbres (RF, XGBoost), un entier ordinal suffit ; les seuils isolent les valeurs sans dépendre de l'ordre.

5.3.3 Normalisation

StandardScaler appliqué uniquement pour la Logistic Regression. Sans normalisation, les coefficients seraient écrasés par les variables à grandes valeurs (co2_estime en milliers contre n_stops entre 0 et 15). Random Forest et XGBoost travaillent sur des seuils ordinaux — l'échelle absolue n'a aucun impact.

Règle appliquée : fit_transform() uniquement sur le train, transform() sur val et test pour éviter toute fuite d'information.

5.3.4 Split 70 / 15 / 15 stratifié

- Train : 99 694 lignes — proportion classe 1 : 8.4%
- Validation : 21 363 lignes — proportion classe 1 : 8.4%
- Test : 21 363 lignes — proportion classe 1 : 8.4%

La stratification garantit que le déséquilibre est identique dans les 3 ensembles.

5.4 Trois modèles candidats

Modèle 1 — Régression Logistique (baseline)

Modèle linéaire : estime la probabilité d'appartenir à la classe 1 via une fonction sigmoïde. Point fort : interprétable directement via ses coefficients — utile pour justifier les prédictions auprès de parties prenantes non techniques. Limite attendue : la frontière de décision est linéaire, ce qui ne colle probablement pas avec la construction de la cible (deux seuils fixes sur deux variables).

Modèle 2 — Random Forest

Ensemble de 100 arbres de décision entraînés en parallèle sur des sous-échantillons aléatoires (bagging). Vote majoritaire pour la prédiction finale. Gère naturellement les non-linéarités et les interactions entre features, sans normalisation. Inconvénient : moins interprétable qu'un modèle linéaire, plus gourmand en mémoire sur de grands ensembles.

Modèle 3 — XGBoost

Boosting gradient : les arbres sont construits séquentiellement, chaque arbre corrige les erreurs du précédent. scale_pos_weight compense le déséquilibre de classes sans sur-échantillonner les données. Algorithme de référence sur les données tabulaires en compétitions et en production industrielle. Plus sensible aux hyperparamètres que Random Forest — d'où l'optimisation par GridSearchCV.

5.5 Métriques d'évaluation

- Accuracy : proportion globale de bonnes prédictions
- F1-score macro : moyenne des F1 par classe, adapté au déséquilibre — métrique principale
- Recall classe 1 : proportion de liaisons substituables correctement détectées — métrique prioritaire pour ObRail (manquer une liaison = opportunité perdue)
- AUC-ROC : capacité discriminante quel que soit le seuil de décision

5.6 Comparaison des modèles sur le jeu de validation

Modèle	Accuracy	F1-macro	Recall cl. 1	Précision cl. 1	AUC-ROC
Logistic Regression	98.47%	0.9539	99.44%	84.95%	0.997

Random Forest	100%	1.000	100%	100%	1.000
XGBoost	99.79%	0.9932	100%	97.55%	1.000

XGBoost obtient les meilleurs scores sur l'ensemble des métriques, notamment sur le Recall classe 1 — critère prioritaire pour ne pas manquer de liaisons substituables. Random Forest est une alternative solide avec des performances proches. La régression logistique reste utile comme baseline interprétable mais ne capte pas les non-linéarités.

5.7 Optimisation par GridSearchCV

Hyperparamètre	Valeurs testées	Meilleure valeur
n_estimators	100, 200	200
max_depth	3, 5, 7	7
learning_rate	0.05, 0.1, 0.2	0.2

18 combinaisons × 5 folds = 90 fits. Meilleur F1-macro en cross-validation : 0.9974.

5.8 Cross-validation — Robustesse du modèle

La cross-validation à 5 folds vérifie que les performances du classifieur ne sont pas dues au hasard d'un découpage train/test particulier. Le modèle est évalué 5 fois sur des sous-ensembles différents. Le F1-score macro est utilisé comme métrique car les classes sont déséquilibrées (91.6% vs 8.4%). Un écart-type faible confirme la stabilité et la capacité de généralisation du modèle.

Fold	F1-score
Fold 1	0.9974
Fold 2	0.9974
Fold 3	0.9974
Fold 4	0.9967
Fold 5	0.9979
Moyenne	0.9974
Écart-type	0.0004 — ✓ Modèle stable

5.9 Résultats finaux sur le jeu de test

Métrique	Résultat
F1-macro final	0.997
AUC-ROC final	1.000
Vrais positifs (liaisons substituables détectées)	1 789 sur 1 792
Faux négatifs (liaisons substituables manquées)	3 — opportunités non détectées

5.10 Feature importance

XGBoost attribue 97.2% de l'importance à distance_km seule — confirmant la circularité avec la variable cible construite sur ce critère. Random Forest est plus équilibré : distance_km (37.0%), consommation_totale (32.1%), duration_minutes (19.8%), co2_estime (5.8%), country (3.8%).

6. API REST — Exposition des modèles

6.1 Architecture et choix techniques

Une API REST unique a été développée avec FastAPI (Python) pour exposer les deux modèles. FastAPI a été choisi pour sa génération automatique de documentation Swagger, sa validation des données via Pydantic, et sa performance native basée sur ASGI.

Les deux modèles sont chargés en mémoire au démarrage du serveur (une seule fois) pour garantir des temps de réponse faibles. En cas d'échec du chargement d'un modèle, l'API reste opérationnelle et retourne un message d'erreur explicite sur la route /health plutôt que de planter entièrement.

6.2 Routes exposées

Route	Méthode	Description
GET /	GET	Message d'accueil et liste des routes disponibles
GET /health	GET	Vérifie que l'API et les deux modèles sont opérationnels
POST /predict/substitution	POST	Classification — prédit si une liaison est substituable à l'avion (0 ou 1)
POST /predict/co2	POST	Régression — estime les émissions CO2 futures selon un scénario
POST /predict	POST	Route de compatibilité v1 — identique à /predict/substitution

6.3 Modèles chargés

- `classification_substitution_avion.joblib` — modèle XGBoost entraîné sur l'enjeu classification. Nécessite `encoders.joblib` (LabelEncoder pour `type_train` et `country`) et `scaler.joblib` (StandardScaler)
- `regression_co2.joblib` — Pipeline sklearn complet (ColumnTransformer + XGBoost) pour l'enjeu régression. Le préprocesseur est embarqué directement dans le fichier — un seul joblib suffit

6.4 Tests réalisés

Test	Paramètres envoyés	Résultat obtenu	Attendu
/health	Aucun	status: ok, 2 modèles ok	✓
Paris → Marseille (800 km)	distance=800, durée=195, pays=FR, electric	substituable=1, probabilité=1.0	✓
Paris → New York (5 800 km)	distance=5800, durée=800, pays=FR, electric	substituable=0, probabilité=0.006	✓
CO2 scénario reference	distance=400, diesel, reference	104.34 kgCO2	✓
CO2 scénario électrification	distance=400, diesel, diesel_50_electrique	64.74 kgCO2 (-38%)	✓

L'ensemble des routes a été testé via l'interface Swagger. Les captures ci-dessous illustrent les résultats obtenus.

Vérification du statut des modèles — GET /health :

The screenshot shows a REST client interface with the following details:

- Curl:** `curl -X 'GET' \ 'http://localhost:8000/health' \ -H 'accept: application/json'`
- Request URL:** `http://localhost:8000/health`
- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "status": "ok",
  "timestamp": "2026-06-07T13:12:21.114050Z",
  "modules": {
    "classification_substitution": "ok",
    "regression_co2": "ok"
  }
}
```
 - Response headers:** (empty)

Figure 1 — Les deux modèles sont opérationnels (status : ok)

Classification substitution avion/train — POST /predict/substitution :

Test 1 : Paris → Marseille (800 km, électrique) — liaison substituable attendue :

The screenshot shows a REST client interface with the following details:

- Curl:** `curl -X 'POST' \ 'http://localhost:8000/predict/substitution' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{
 "distance_km": 800,
 "duration_minutes": 195,
 "n_stops": 3,
 "co2_estimate": 450000,
 "consumation_totale": 16000,
 "type_train": "electric",
 "country": "FR"
}'`
- Request URL:** `http://localhost:8000/predict/substitution`
- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "substitution_avion": 1,
  "probabilite": 1,
  "label": "Substituable à l'avion"
}
```
 - Response headers:**

```
content-length: 76
content-type: application/json
date: Sun, 07 Jun 2026 13:14:51 GMT
server: uvicorn
```

Figure 2 — Résultat : `substitution_avion = 1`, `probabilité = 1.0` (Substituable à l'avion)

Test 2 : Paris → New York (5 800 km) — liaison non substituable attendue :

The screenshot shows a REST client interface with the following details:

- Curl:** `curl -X 'POST' \ 'http://localhost:8000/predict/substitution' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{
 "distance_km": 5800,
 "duration_minutes": 800,
 "n_stops": 1,
 "co2_estimate": 900000,
 "consumation_totale": 90000,
 "type_train": "electric",
 "country": "FR"
}'`
- Request URL:** `http://localhost:8000/predict/substitution`
- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "substitution_avion": 0,
  "probabilite": 0.0056,
  "label": "Non substituable"
}
```
 - Response headers:**

```
content-length: 72
content-type: application/json
date: Sun, 07 Jun 2026 13:16:03 GMT
server: uvicorn
```

Figure 3 — Résultat : `substitution_avion = 0`, `probabilité = 0.0056` (Non substituable)

Le modèle distingue correctement les deux cas : une liaison de 800 km avec une durée raisonnable est identifiée comme substituable, tandis qu'une liaison transatlantique de 5 800 km est immédiatement rejetée avec une probabilité quasi nulle.

Régression CO2 — POST /predict/co2 :

Deux scénarios sont comparés sur la même liaison (Paris → Lyon, train diesel, 400 km) pour démontrer l'impact des politiques de développement du réseau :

Scénario reference — aucune évolution du réseau :

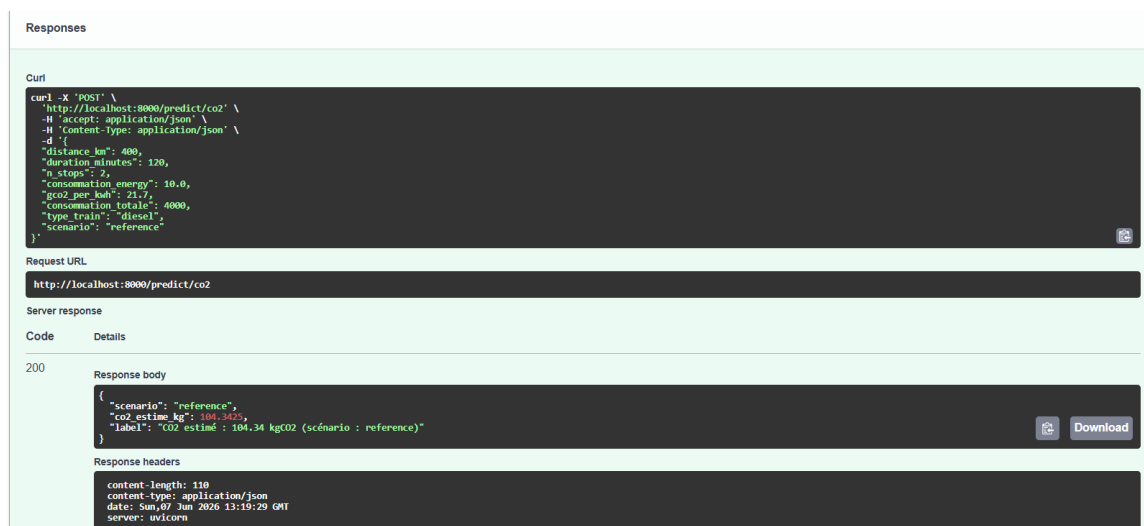


Figure 4 — CO2 estimé : 104.34 kgCO2 (scénario de référence)

Scénario diesel_50_electrique — électrification à 50% des trains diesel :



Figure 5 — CO2 estimé : 64.74 kgCO2 (scénario électrification partielle)

L'électrification partielle des trains diesel permet une réduction de 38% des émissions CO2 sur cette liaison (104.34 → 64.74 kgCO2). Ce type de simulation est directement exploitable par les décideurs européens pour évaluer l'impact environnemental de différentes politiques ferroviaires.

6.5 Métriques de monitoring identifiées pour la production

- Taux de requêtes par minute par route — détection de surcharge
- Distribution des prédictions dans le temps — détecter une dérive (proportion de 1 sur /predict/substitution qui change anormalement, moyenne kgCO2 sur /predict/co2)
- Data drift — comparer les distributions des features en entrée vs le jeu d'entraînement à l'aide d'Evidently AI ou WhyLogs
- Latence P50 / P95 / P99 par route — garantir les temps de réponse
- Taux d'erreurs HTTP 422 (payload invalide) et 500 (erreur modèle)

7. Pipeline CI/CD et Tests automatisés

7.1 Contexte et objectif

Suite à la première soutenance, un pipeline d'intégration continue a été mis en place pour automatiser les tests de l'API à chaque modification du code. L'objectif est de garantir que chaque commit pousse sur GitHub ne casse pas les fonctionnalités existantes de l'API, sans avoir à tester manuellement à chaque fois.

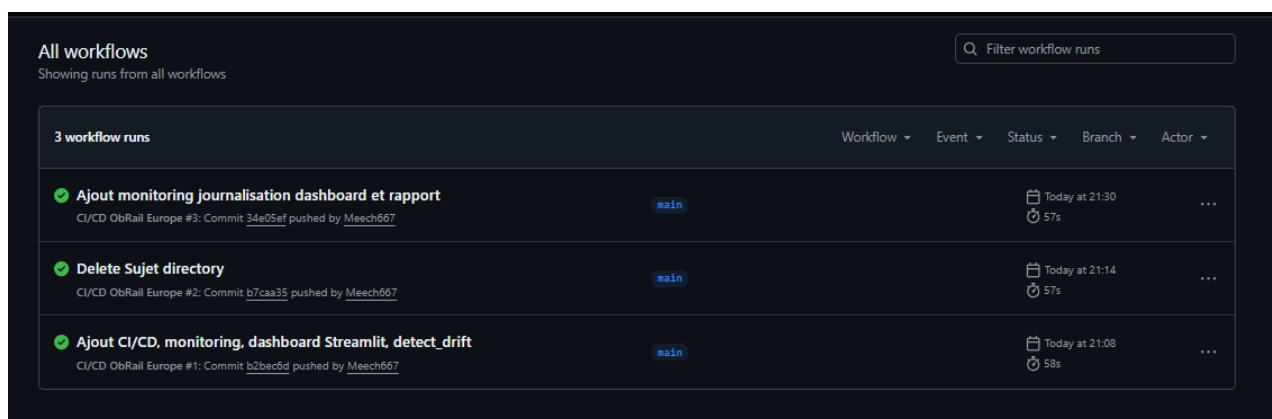
Le pipeline est déclenché automatiquement à chaque push ou pull request sur la branche main. Il installe les dépendances, lance les tests et affiche le résultat dans l'onglet Actions de GitHub.

7.2 Architecture du pipeline

Le fichier `.github/workflows/ci.yml` définit le pipeline GitHub Actions. Il se compose de trois étapes principales : checkout du code, installation des dépendances depuis `requirements.txt`, et exécution des tests avec rapport de couverture.

Étape	Action	Détail
1 - Checkout	Récupération du code	actions/checkout@v4
2 - Setup Python	Installation Python 3.10	actions/setup-python@v5
3 - Dépendances	pip install -r requirements.txt	Inclut pytest, httpx, pytest-cov
4 - Tests	pytest tests/ --cov=api	5 tests avec rapport de couverture

Le pipeline utilise TestClient de FastAPI pour simuler les requêtes HTTP sans lancer de serveur réel. Si les fichiers joblib sont absents, les tests sont automatiquement ignorés via `conftest.py` sans faire échouer le pipeline.



The screenshot shows the 'All workflows' page on GitHub. It displays three workflow runs, all with a green status icon indicating success. The runs are:

- Ajout monitoring journalisation dashboard et rapport**: CI/CD ObRail Europe #3: Commit 34e05ef pushed by Meech667. Status: main. Run time: Today at 21:30, 57s.
- Delete Sujet directory**: CI/CD ObRail Europe #2: Commit b7caa35 pushed by Meech667. Status: main. Run time: Today at 21:14, 57s.
- Ajout CI/CD, monitoring, dashboard Streamlit, detect_drift**: CI/CD ObRail Europe #1: Commit b2bec6d pushed by Meech667. Status: main. Run time: Today at 21:08, 58s.

7.3 Tests automatisés

Cinq tests couvrent l'ensemble des routes principales de l'API et vérifient la cohérence des prédictions, pas seulement que l'API démarre.

Test	Route	Ce qu'il vérifie
------	-------	------------------

test_health	GET /health	Réponse 200 et les 2 modèles chargés
test_predict_substitution_substituable	POST /predict/substitution	Paris-Marseille 800km → substituable = 1
test_predict_substitution_non_substituable	POST /predict/substitution	5 800km → non substituable = 0
test_predict_co2_reference	POST /predict/co2	Scénario reference retourne un float positif
test_predict_co2_electrification	POST /predict/co2	CO2 electrification < CO2 reference

-> Le dernier test verifie que la logique métier est correcte : l'électrification doit réduire les émissions. Ce n'est pas un simple test de format mais un test de cohérence de la prédiction.

```
tests/test_api.py::test_health PASSED [ 20%]
tests/test_api.py::test_predict_substitution_substituable PASSED [ 40%]
tests/test_api.py::test_predict_substitution_non_substituable PASSED [ 60%]
tests/test_api.py::test_predict_co2_reference PASSED [ 80%]
tests/test_api.py::test_predict_co2_electrification PASSED [100%]
```

7.4 Résultats

- 5 tests passent au vert sur le pipeline GitHub Actions
- Couverture de code api.py mesurée à chaque run
- Badge CI/CD visible en haut du README - preuve visuelle instantanée
- Déclenchement automatique sur chaque push - 3 runs enregistrés dans l'historique Actions

8. Monitoring, journalisation et détection de dérive

8.1 Architecture du monitoring

Un système de monitoring complet a été développé pour surveiller l'API en continu, détecter automatiquement les incidents et alimenter la feedback loop MLOps. Il se compose de quatre composants complémentaires, tous conformes au RGPD - seules les features numériques ferroviaires sont enregistrées, aucune donnée personnelle n'est loggée.

Composant	Fichier	Rôle
Journalisation	logs/api.log	Enregistre chaque requête : route, features, résultat, durée
Monitoring	monitoring/monitoring.py	Calcule métriques, détecte incidents, génère rapport
Détection drift	monitoring/detect_drift.py	Compare features entrantes vs statistiques entraînement
Dashboard	monitoring/dashboard.py	Tableau de bord Streamlit temps réel

8.2 Journalisation

Chaque requête reçue par l'API est enregistrée dans logs/api.log avec le format suivant : timestamp, niveau de log, route appelée, features numériques reçues, résultat retourné et durée de traitement en millisecondes.

Exemple de ligne de log :

```
2026-06-29 18:32:14 - INFO - REQUETE route=/predict/substitution distance_km=800.0 duration_minutes=195.0 n_stops=3  
resultat=1 proba=1.0 duree_ms=14.0
```

api.log	
C:\Users> sato > Desktop > epsi > Info > MSPR > Mspr 2 et 4 > MSPR 622 > logs > api.log	
1 2026-06-29 18:29:20,617 - INFO - Modèle classification_substitution_avion chargé avec succès	
2 2026-06-29 18:29:20,839 - INFO - Modèle regression_co2 chargé avec succès	
3 2026-06-29 18:29:26,586 - INFO - Modèle classification_substitution_avion chargé avec succès	
4 2026-06-29 18:29:26,719 - INFO - Modèle regression_co2 chargé avec succès	
5 2026-06-29 18:32:14,168 - INFO - REQUETE route=/predict distance_km=1200.0 duration_minutes=360.0 n_stops=2 co2_estime=450000.0 consommation_totale=20000.0 resulta	
6 2026-06-29 18:32:22,102 - INFO - REQUETE route=/predict/substitution distance_km=1200.0 duration_minutes=360.0 n_stops=2 co2_estime=450000.0 consommation_totale=20	
7 2026-06-29 18:32:30,867 - INFO - REQUETE route=/predict/co2 distance_km=400.0 duration_minutes=120.0 n_stops=2 consommation_energy=10.0 gco2_per_kwh=21.7 consommat	
8 2026-06-29 18:32:41,406 - INFO - REQUETE route=/health statut_clf=True statut_co2=True duree_ms=0.0	
9 2026-06-29 18:32:43,437 - INFO - REQUETE route=/health statut_clf=True statut_co2=True duree_ms=0.0	
10 2026-06-29 20:56:15,407 - INFO - Modèle classification_substitution_avion chargé avec succès	
11 2026-06-29 20:56:15,551 - INFO - Modèle regression_co2 chargé avec succès	
12 2026-06-29 21:00:15,731 - INFO - Modèle classification_substitution_avion chargé avec succès	
13 2026-06-29 21:00:15,879 - INFO - Modèle regression_co2 chargé avec succès	
14 2026-06-29 21:00:26,269 - INFO - Modèle classification_substitution_avion chargé avec succès	
15 2026-06-29 21:00:26,415 - INFO - Modèle regression_co2 chargé avec succès	
16 2026-06-29 22:54:20,096 - INFO - Modèle classification_substitution_avion chargé avec succès	
17 2026-06-29 22:54:20,399 - INFO - Modèle regression_co2 chargé avec succès	
18 2026-06-29 22:55:42,769 - INFO - Modèle classification_substitution_avion chargé avec succès	
19 2026-06-29 22:55:42,823 - INFO - Modèle regression_co2 chargé avec succès	
20 2026-06-29 22:56:14,118 - INFO - Modèle classification_substitution_avion chargé avec succès	
21 2026-06-29 22:56:14,199 - INFO - Modèle regression_co2 chargé avec succès	
22 2026-06-29 22:56:16,403 - INFO - Modèle classification_substitution_avion chargé avec succès	
23 2026-06-29 22:56:16,468 - INFO - Modèle regression_co2 chargé avec succès	
24 2026-06-29 22:56:37,814 - INFO - REQUETE route=/health statut_clf=True statut_co2=True duree_ms=0.0	
25 2026-06-29 22:56:46,238 - INFO - REQUETE route=/predict distance_km=1200.0 duration_minutes=360.0 n_stops=2 co2_estime=450000.0 consommation_totale=20000.0 resulta	
26 2026-06-29 22:56:53,835 - INFO - REQUETE route=/predict/substitution distance_km=1200.0 duration_minutes=360.0 n_stops=2 co2_estime=450000.0 consommation_totale=20	
27 2026-06-29 22:57:02,004 - INFO - REQUETE route=/predict/co2 distance_km=400.0 duration_minutes=120.0 n_stops=2 consommation_energy=10.0 gco2_per_kwh=21.7 consommat	
28	

Les démarrages de l'API sont également loggés, permettant de suivre les redémarrages et de détecter des instabilités éventuelles.

8.3 Métriques et détection automatique d'incidents

Le script `monitoring/monitoring.py` lit le fichier `api.log` et calcule les métriques suivantes : nombre total de requêtes par route, taux d'erreur, durée moyenne de traitement, distribution des prédictions de substitution et moyenne des CO2 prédits.

Des alertes automatiques sont déclenchées sans intervention humaine dans trois cas :

- Taux d'erreur supérieur à 5% - surcharge ou bug détecté
- Durée moyenne supérieure à 2 secondes - dégradation des performances
- Proportion de liaisons substituables hors de la plage 10-90% - anomalie de distribution

```
(base) PS C:\Users\satou> cd "C:\Users\satou\Desktop\epsi\Info\MSPR\Mspr 2 et 4\MSPR 622"
(base) PS C:\Users\satou\Desktop\epsi\Info\MSPR\Mspr 2 et 4\MSPR 622> python monitoring/monitoring.py
Analyse des logs ObRail Europe...

=====
RAPPORT DE MONITORING - ObRail Europe
Généré le : 2026-06-29 22:59:31
=====

--- MÉTRIQUES GÉNÉRALES ---
Total requêtes traitées : 9
Total erreurs           : 0
Taux d'erreur           : 0.0%

--- REQUÊTES PAR ROUTE ---
/health                 3 requête(s)
/predict                2 requête(s)
/predict/co2            2 requête(s)
/predict/substitution    2 requête(s)

--- DURÉE MOYENNE PAR ROUTE (ms) ---
/health                 0.0 ms
/predict                45.8 ms
/predict/co2            20.1 ms
/predict/substitution    9.5 ms

--- PRÉDICTIONS /predict/substitution ---
Prédictions totales    : 4
```

8.4 Détection de dérive et feedback loop MLOps

Le script `detect_drift.py` compare les distributions des features reçues dans les dernières requêtes avec les statistiques de référence calculées sur le jeu d'entraînement. Si une feature dépasse 2 écarts-types par rapport à la référence, une alerte est générée et un rapport est écrit dans `logs/drift_report.txt`.

Feature	Moyenne entraînement	Écart-type	Seuil d'alerte
distance_km	99.6 km	142.4	Dérive > 2 sigma
duration_minutes	95.4 min	82.3	Dérive > 2 sigma
n_stops	14.2	9.5	Dérive > 2 sigma
co2_estime	251 165 gCO2	539 481	Dérive > 2 sigma

Ce mécanisme constitue le premier maillon de la feedback loop MLOps : si une dérive est détectée, c'est le signal que le modèle reçoit des données qui s'éloignent de son contexte d'entraînement et qu'un réentraînement est nécessaire. En production, Evidently AI ou WhyLogs automatiseraient cette détection avec des rapports HTML et des intégrations CI/CD.

-> Prometheus collecterait les métriques en temps réel, Grafana les visualiserait sur un tableau de bord opérationnel, et Loki centraliserait les logs applicatifs. Ces outils s'intègrent naturellement en remplacement de notre stack actuelle pour une mise en production institutionnelle.

```
(base) PS C:\Users\satou\Desktop\epsi\Info\MSPR\Mspr 2 et 4\MSPR 622> python monitoring/detect_drift.py
Détection de drift - ObRail Europe...

=====
RAPPORT DÉTECTION DRIFT - ObRail Europe
Généré le      : 2026-06-29 23:01:20
Requêtes analysées : 4
=====

--- ANALYSE PAR FEATURE ---

distance_km
Référence (entraînement) : moy=99.6,  $\sigma$ =142.4
Statut                   : Insuffisant (4 < 5 requis)

duration_minutes
Référence (entraînement) : moy=95.4,  $\sigma$ =82.3
Statut                   : Insuffisant (4 < 5 requis)

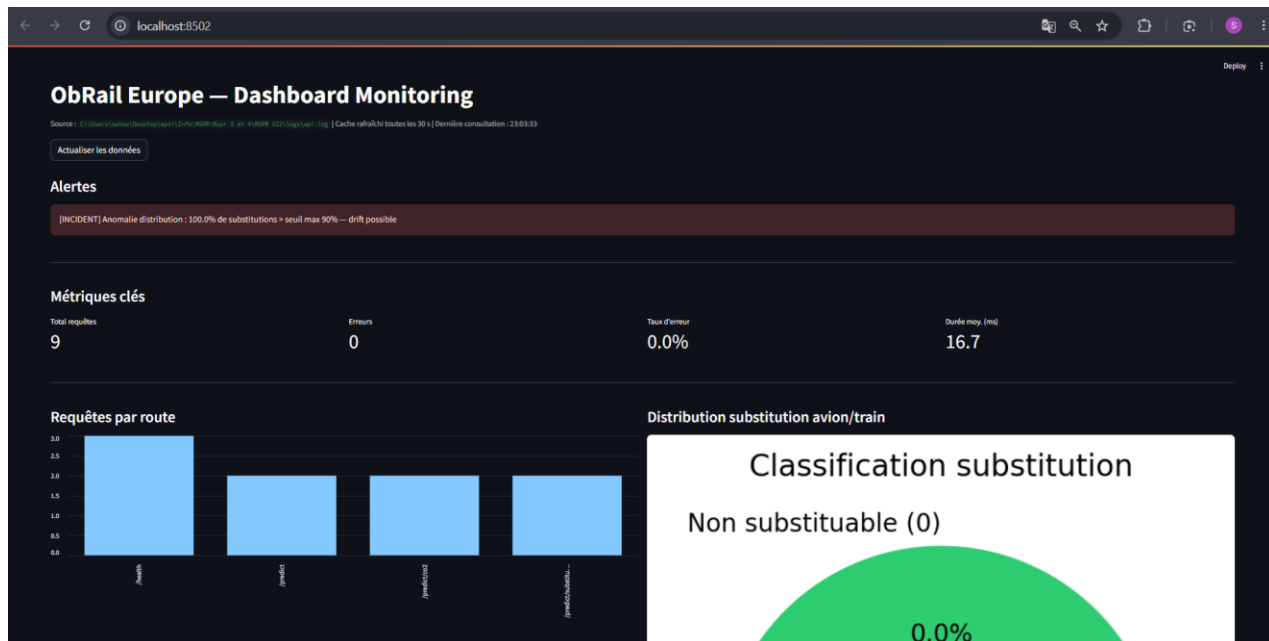
n_stops
Référence (entraînement) : moy=14.2,  $\sigma$ =9.5
Statut                   : Insuffisant (4 < 5 requis)

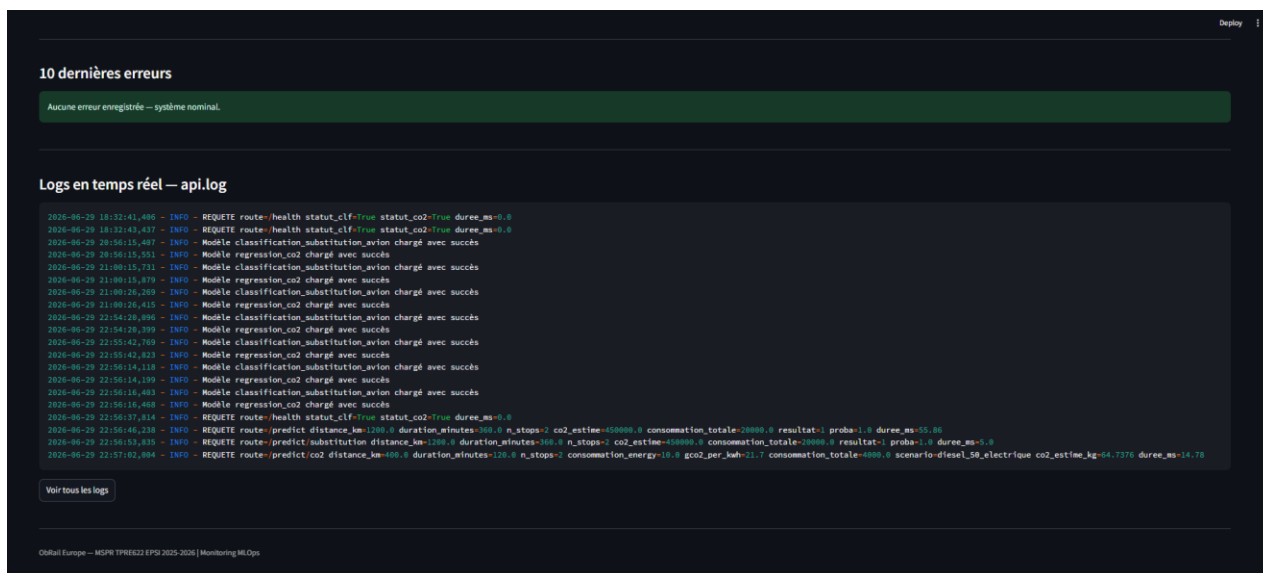
co2_estime
Référence (entraînement) : moy=251165.0,  $\sigma$ =539481.0
Statut                   : Insuffisant (4 < 5 requis)

--- ALERTES DRIFT ---
Aucun drift détecté - distributions stables.
```

8.5 Dashboard Streamlit

Un tableau de bord visuel a été développé avec Streamlit et se lance avec la commande `streamlit run monitoring/dashboard.py`. Il affiche en temps réel les métriques clés, la distribution des prédictions, la timeline des requêtes, le tableau des dernières erreurs et les logs bruts de api.log.





8.6 Documentation et incidents

Deux documents accompagnent le système de monitoring. MONITORING.md décrit l'architecture complète, le format des logs, la politique RGPD, les commandes de lancement et le lien avec la feedback loop MLOps. INCIDENTS.md documente deux incidents techniques réels rencontrés pendant le développement.

Incident	Symptôme	Cause	Solution
encoders.joblib manquant	API plante au démarrage avec FileNotFoundError	Modèle classification nécessite encoders.joblib séparé	Chargement conditionnel avec message d'erreur explicite sur /health
Cross-validation absente notebook 01	Notebook régression sans cellule cross-validation	Cross-validation générée uniquement dans le notebook 02	Ajout manuel de la cellule cross_val_score dans notebook 01

9. Benchmark des services d'intelligence artificielle

Dans le cadre du projet ObRail Europe, nous avons évalué trois plateformes cloud majeures proposant des services de machine learning automatisé (AutoML) : Amazon SageMaker, Google Vertex AI et Microsoft Azure Machine Learning. L'objectif était de déterminer si l'une de ces solutions aurait été plus adaptée que notre approche open source pour répondre aux deux enjeux du projet — la classification des liaisons substituables et la régression des émissions CO2.

9.1 Amazon SageMaker (AWS)

SageMaker est la plateforme ML d'Amazon Web Services, lancée en 2017. Elle est aujourd'hui la plus utilisée au monde avec environ 34% de parts de marché. Elle propose SageMaker Autopilot, son module AutoML, qui analyse automatiquement les données, sélectionne les algorithmes et optimise les hyperparamètres sans écrire une ligne de code.

Capacités techniques — Autopilot prend en charge la régression, la classification binaire et la classification multi-classes sur des données tabulaires — les deux tâches exactes de notre projet. Il génère automatiquement les notebooks Python montrant comment le modèle a été construit, ce qui apporte une certaine transparence. La plateforme supporte scikit-learn, XGBoost, TensorFlow et PyTorch en mode personnalisé.

Prix — Le modèle de tarification est entièrement à l'usage, sans abonnement fixe. Une instance de notebook de base coûte environ 0.05\$/heure. Une instance GPU commence à 0.74\$/heure. Un déploiement moyen avec un modèle en production tourne autour de 260\$/mois en cumulant l'entraînement, l'endpoint d'inférence et le stockage. Une période d'essai gratuite de 2 mois seulement est proposée.

RGPD — Des régions européennes existent (Paris, Francfort, Stockholm) permettant de garantir que les données ne quittent pas l'Europe, mais cela demande une configuration explicite.

Pertinence pour ObRail — Pertinent techniquement, mais surdimensionné pour un dataset de 142 000 lignes. La complexité d'administration et le coût récurrent ne se justifient pas pour un projet de recherche ferroviaire à cette échelle.

9.2 Google Vertex AI

Vertex AI est la plateforme unifiée de machine learning de Google Cloud, qui intègre l'ancien service AutoML Tables. Elle détient environ 22% du marché des plateformes ML. Son module AutoML Tabular est conçu spécifiquement pour les tâches supervisées sur données tabulaires — classification et régression.

Capacités techniques — Vertex AI AutoML Tabular gère l'ingestion depuis BigQuery ou Cloud Storage, applique le prétraitement automatique (valeurs manquantes, encodage des catégories), entraîne et optimise les modèles, et propose des explications via Vertex AI Explainability. C'est la plateforme la plus propre visuellement et la plus facile à prendre en main selon les retours utilisateurs.

Prix — L'entraînement de modèles de classification ou régression sur données tabulaires coûte 21.25\$ par node-heure. Un endpoint de classification déployé en continu coûte 1.375\$/heure — soit environ 990\$ par mois si on le laisse tourner 30 jours même sans aucune requête. Google offre 300\$ de crédits gratuits pour les nouveaux utilisateurs, valables 90 jours.

RGPD — Google Cloud dispose de régions européennes (Belgique, Pays-Bas, Francfort). La résidence des données en Europe est configurable, mais Google reste une entreprise américaine soumise au Cloud Act — point sensible pour des données institutionnelles européennes travaillant avec la Commission européenne.

Pertinence pour ObRail — Techniquement excellent et le plus accessible des trois. Mais le coût de l'endpoint actif en permanence est prohibitif pour un usage institutionnel non continu. Le Cloud Act pose aussi une question éthique réelle pour un observatoire travaillant avec le Parlement européen.

9.3 Microsoft Azure Machine Learning

Azure ML est la plateforme de machine learning de Microsoft, détenant environ 29% du marché. Elle propose un mode AutoML complet avec une interface no-code (Designer) et un accès code via SDK Python.

Capacités techniques — L'AutoML d'Azure ML teste automatiquement plusieurs algorithmes (régression linéaire, Random Forest, XGBoost, LightGBM) sur les données fournies et produit un tableau comparatif des modèles — exactement ce que nous avons fait manuellement. Les clusters de calcul (AmlCompute) scalent dynamiquement selon la charge.

Prix — Azure ML ne facture pas de frais de plateforme fixes. Le coût est uniquement lié aux ressources de calcul consommées (VMs Azure), ce qui le rend plus prévisible que SageMaker. Une VM standard CPU coûte entre 0.05 et 0.20\$/heure selon la taille.

RGPD — C'est le point fort d'Azure pour ObRail. Microsoft dispose de datacenters en France (Paris, Marseille), garantissant une résidence des données sur le sol français. Azure est certifié ISO 27001, SOC 2, et conforme aux exigences RGPD de manière native. C'est la meilleure option réglementaire pour un organisme travaillant avec les institutions européennes.

Pertinence pour ObRail — Le plus pertinent des trois sur le plan réglementaire. Mais la courbe d'apprentissage reste élevée et la valeur ajoutée par rapport à scikit-learn reste faible pour un dataset de cette taille.

9.4 Tableau comparatif synthétique

Critère	AWS SageMaker	Google Vertex AI	Azure ML
Prix entraînement	0.05\$/h (CPU)	21.25\$/node-heure	Selon VM, 0.05-0.20\$/h
Prix inférence	0.23\$/h (endpoint)	1.375\$/h (endpoint actif)	Pas de frais fixes
Essai gratuit	2 mois (limité)	300\$ / 90 jours	200\$ / 30 jours
AutoML tabulaire	✓ Autopilot	✓ AutoML Tables	✓ AutoML intégré
Explicabilité	✓ SHAP intégré	✓ Vertex Explainability	✓ SHAP + glass-box
RGPD / résidence EU	Configurable	Cloud Act US	✓ Datacenter France
Vendor lock-in	Fort (AWS only)	Fort (GCP only)	Fort (Azure only)
Part de marché ML	34%	22%	29%

9.5 Pourquoi l'approche open source a été retenue

- Coût nul — 142 000 lignes et 7 features ne justifient pas 21\$/heure pour AutoML Vertex AI ou 165\$/mois pour un endpoint SageMaker
- Contrôle total — chaque choix algorithmique est documenté et justifié dans les notebooks, conformément aux exigences du cahier des charges
- RGPD natif — les données ne quittent jamais le poste de travail, pas d'exposition au Cloud Act américain
- Compatibilité directe — joblib chargé dans FastAPI sans SDK propriétaire ni dépendance cloud
- Liberté algorithmique — les 3 familles demandées (linéaire, bagging, boosting) testées et comparées explicitement

10. Veille technique, risques et recommandations

10.1 Veille algorithmique

Les modèles de gradient boosting comme XGBoost sont adaptés aux données tabulaires, car ils captent les relations non linéaires tout en restant plus explicables qu'un réseau de neurones profond.

La tendance actuelle dans le domaine du ML tabulaire va vers des modèles comme LightGBM (plus rapide que XGBoost sur les grands datasets) et CatBoost (meilleure gestion native des variables catégorielles sans encodage manuel). Une évolution naturelle du projet serait de les tester comme modèles candidats supplémentaires lors d'une V2. Des travaux récents de 2024-2025 montrent également que les TabNet (réseaux de neurones spécialisés pour le tabulaire) commencent à rivaliser avec le boosting sur certains cas d'usage, mais leur coût computationnel reste élevé pour un gain marginal sur ce type de données ferroviaires.

10.2 Veille MLOps

La solution doit prévoir un monitoring : logs API, latence, erreurs, dérive des données et suivi des prédictions. MLflow, Evidently AI ou Prometheus peuvent être ajoutés ensuite.

MLflow permettrait de tracer chaque expérience — hyperparamètres, métriques, versions du modèle — et de comparer les runs automatiquement. C'est le standard open source du suivi d'expériences ML, gratuit et déployable en local.

Evidently AI est spécialisé dans la détection de dérive des données (data drift). Concrètement : si dans 6 mois les liaisons ferroviaires européennes évoluent (nouvelles lignes, changements tarifaires, nouveaux opérateurs), les distributions des features en entrée de l'API vont changer par rapport au jeu d'entraînement. Evidently génère des rapports HTML automatiques qui signalent ces dérives avant que les prédictions ne se dégradent.

10.3 Conformité RGPD

Le jeu de données ne contient pas de données personnelles directes. Le projet doit tout de même garder la transparence des sources, la documentation des traitements et la minimisation des données.

- Transparence — les sources des données sont documentées et traçables (Back-on-Track, GTFS, Eurostat, ADEME). Tout tiers peut vérifier l'origine des informations utilisées pour entraîner les modèles
- Minimisation — seules les colonnes nécessaires aux enjeux ont été retenues. Les coordonnées GPS sont utilisées uniquement pour calculer la distance haversine et ne sont pas exposées via l'API
- Documentation — le processus ETL, les transformations appliquées et les choix algorithmiques sont documentés dans les notebooks et ce rapport

10.4 Risques identifiés

Risque 1 — Données hétérogènes selon les pays : Les 4 pays couverts (FR, ES, IT, DE) n'ont pas la même densité de données. L'Italie est sous-représentée dans les liaisons substituables, l'Allemagne domine en volume de trajets courts. Un modèle entraîné sur ce déséquilibre géographique pourrait mal généraliser à d'autres pays européens.

Risque 2 — Cible métier parfois dérivée de règles : Les deux variables cibles sont construites artificiellement. La circularité entre features d'entrée et cible explique les performances quasi parfaites — en conditions réelles avec des données mesurées terrain, les performances seraient inférieures.

Risque 3 — Outliers réels sur les longs trajets : Certains trajets dépassent 20h (duration_minutes > 1200). Ces outliers sont réels (trains de nuit transcontinentaux) et non des erreurs — ils influencent les distributions mais ont été conservés.

Risque 4 — Manque de données économiques ou politiques futures : Le prix du billet est probablement le facteur décisif dans le choix avion/train pour un voyageur réel. Le modèle de classification ignore totalement cette dimension faute de données disponibles.

10.5 Recommandations

Conserver la solution interne open source pour la V1, ajouter MLflow, prévoir un monitoring de dérive et réentraîner les modèles à chaque grande mise à jour du référentiel.

- V1 actuelle : conserver l'approche open source, coût nul, contrôle total, aucune dépendance cloud
- V2 : intégrer MLflow pour le suivi des expériences et le versioning des modèles
- V2 : ajouter Evidently AI pour surveiller la dérive des données entrantes dans l'API
- V2 : enrichir le dataset avec des données tarifaires (Rail Europe, Trainline API) pour améliorer la pertinence métier
- V2 : étendre la couverture géographique à 10-15 pays européens supplémentaires
- V2 : déploiement cloud sur Azure ML (datacenter France) pour garantir la résidence des données

11. Limites et perspectives

11.1 Limites identifiées

- Variable cible construite par règles métier — la colonne substitution_avion et la colonne kgCO2_futur sont toutes deux créées artificiellement. Cette circularité entre features d'entrée et cible explique les performances quasi parfaites. En conditions réelles sans données de réservation multimodales, les modèles seraient moins performants.
- Couverture géographique limitée — 4 pays seulement (FR, ES, IT, DE). Le modèle n'est pas directement généralisable à toute l'Europe sans réentraînement sur de nouvelles données.
- Absence de données tarifaires — le prix du billet est ignoré faute de données disponibles, ce qui limite la pertinence économique des prédictions.
- Absence de dimension temporelle — la saisonnalité, les jours de semaine et les horaires ne sont pas exploités dans les modèles actuels.
- Déséquilibre géographique — l'Allemagne domine en volume de trajets courts, ce qui peut biaiser les prédictions sur d'autres contextes nationaux.

11.2 Perspectives

- Intégration de données de réservation multimodales pour construire de vraies étiquettes de substitution mesurées sur le terrain
- Extension à 15+ pays européens avec données GTFS Suisse, Belgique, Pays-Bas, Pologne, Autriche
- Ajout d'une dimension temporelle pour capter la saisonnalité et les variations de fréquentation
- Déploiement cloud sur Azure ML (datacenter France) pour production institutionnelle
- Pipeline MLOps complet avec réentraînement automatique déclenché par la détection de drift Evidently AI
- Développement d'une interface utilisateur (Streamlit ou Dash) pour rendre les prédictions accessibles aux partenaires non techniques d'ObRail

12. Conclusion

Ce projet a permis de développer, évaluer et déployer deux modèles d'IA répondant aux enjeux stratégiques d'ObRail Europe : l'estimation des émissions CO2 futures selon des scénarios de développement du réseau ferroviaire, et l'identification automatique des liaisons candidates à remplacer un vol aérien intra-européen.

La démarche suivie est rigoureuse et conforme aux exigences du cahier des charges : exploration et nettoyage des données, feature engineering, comparaison de 3 familles algorithmiques (linéaire, bagging, boosting), optimisation par GridSearchCV, validation croisée à 5 folds, évaluation finale sur jeu de test non biaisé, sauvegarde en joblib et exposition via API REST documentée.

Les deux modèles XGBoost retenus atteignent des performances quasi parfaites ($R^2 = 1.000$ en régression, $F1 = 0.997$ en classification), validées par cross-validation stable (écart-type < 0.001). Ces performances s'expliquent en partie par la circularité entre features et variables cibles construites — limite assumée, documentée et à prendre en compte pour une mise en production réelle.

Le benchmark des trois services IA cloud confirme que l'approche open source était la plus adaptée pour ce projet : coût nul, contrôle algorithmique complet, données non exposées à des serveurs tiers, conformité RGPD native, et compatibilité directe avec l'infrastructure FastAPI développée.

Livrable	État	Localisation
Notebooks d'analyse (régression + classification)	✓ Complet	01_regression_co2.ipynb, 02_classification_substitution_avion.ipynb
Notebooks EDA et cadrage	✓ Complet	eda/00_cadrage_eda.ipynb, eda/01_modelisation_resultats.ipynb
Dossier projet structuré + requirements.txt	✓ Complet	Racine du repo GitHub
Scripts entraînement + tableau comparatif des modèles	✓ Complet	Dans les notebooks (sections comparaison)
Rapport d'évaluation + 17 visualisations	✓ Complet	figures/CLF/ et figures/REG/
Tableau comparatif services IA (benchmark)	✓ Complet	Section 9 de ce rapport
Modèles sauvegardés (4 fichiers joblib)	✓ Complet	models/
Scripts de prédiction standalone	✓ Complet	predict.py, predict_co2.py
API REST fonctionnelle et testée	✓ Complet	api.py — 4 routes documentées
Veille et recommandations	✓ Complet	Section 10 de ce rapport
Rapport technique complet	✓ Ce document	—
Dépôt GitHub public	✓ Complet	https://github.com/Meech667/obrail-europe
Dataset	✓ Complet	https://drive.google.com/file/d/1vt7VH9y0hb1HF0VJOSGWkntRVcbhafPr/view?usp=sharing
Tests automatisés (5 tests pytest)	✓ Complet	tests/test_api.py
Pipeline CI/CD GitHub Actions	✓ Complet	.github/workflows/ci.yml

Monitoring et journalisation	✓ Complet	monitoring/
Gestion de projet Trello	✓ Complet	Board Trello ObRail Europe

Gestion de projet

Méthodologie

Le projet a été conduit selon une approche agile légère, adaptée à une équipe de 4 apprenants travaillant en parallèle sur des enjeux complémentaires. Les décisions techniques ont été prises collectivement lors de points réguliers, et les livrables ont été versionnés sur GitHub avec un historique de commits structuré en 5 phases cohérentes.

Répartition des tâches

Chaque membre a contribué à l'ensemble du projet tout en ayant des responsabilités principales sur une partie spécifique. Cette organisation a permis à tout le monde de maîtriser l'intégralité de la chaîne - des données à l'API - tout en avançant efficacement en parallèle.

Membre	Responsabilité principale	Contributions transverses
Akram	CI/CD, tests pytest, monitoring, dashboard	Classification, GitHub, rapport
Messa	Régression CO2, notebooks, detect_drift	Modélisation, documentation
Adam	Feature engineering, EDA, cadrage	Encodage, dataset Drive
Aymen	API FastAPI, scripts predict, organisation monitoring	Tests Swagger, rapport

Suivi avec Trello

Un tableau Trello a été utilisé pour suivre l'avancement des tâches sur les deux phases du projet. Les cartes sont organisées en trois listes - Terminé, En cours, Backlog - avec des labels colorés par membre pour visualiser la répartition des responsabilités.

Backlog

5

→ ← ...

Adam

Améliorer la documentation de réentraînement

🕒 1 juil. ⌵

Akram

Ajouter badge CI/CD dans le README

🕒 28 juin ⌵

Messa

Vérifier couverture de code pytest

🕒 29 juin ⌵

Akram

Push de test sur GitHub

🕒 2 juil. ⌵

Messa

Adam

Aymen

Akram

Révision orale — CI/CD et monitoring

🕒 2 juil. ⌵

+ Ajouter une carte



En cours

3

→ ← ...

Messa

Adam

Aymen

Akram

☐ Mise à jour rapport sections CI/CD et monitoring

🕒 26 juin ⌵

Messa

Adam

Aymen

Akram

Mise à jour slides CI/CD et monitoring

🕒 27 juin ⌵

Messa

Adam

Aymen

Akram

Gestion de projet Trello

🕒 25 juin ⌵

+ Ajouter une carte



Terminé

36

→ ← ...

Messa

Adam

Aymen

Akram

Chargement et exploration du dataset eu_trips_v2.csv

🕒 30 mai ⌵

Adam

Nettoyage des données

🕒 30 mai ⌵

Adam

Feature engineering

🕒 31 mai ⌵

Messa

Adam

Aymen

Akram

Analyse exploratoire et heatmap de corrélation

🕒 31 mai ⌵

Messa

Adam

Aymen

Akram

Split 70/15/15 stratifié

🕒 1 juil. ⌵

+ Ajouter une carte

